

Synchronous Inter-Service Communication in Microservices Architecture

 systemdesignschool.io/fundamentals/synchronous-communication

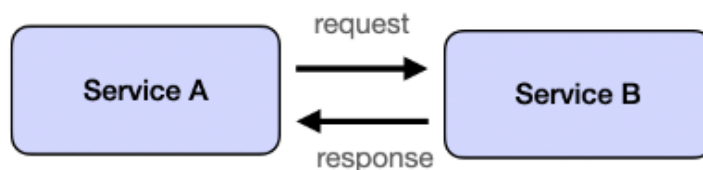


Synchronous Communication

We explored two communication patterns for microservices: synchronous and asynchronous. Synchronous communication provides immediate responses through request-response interactions. This article examines when blocking calls work well and when they cause problems.

How Synchronous Communication Works

A client sends a request and waits for a response before continuing. The client blocks during this time. It cannot process other work until it receives an answer or the request times out.



An API gateway receives a user request for order details. It calls the order service with order ID 12345. The gateway thread waits. The order service queries its database, finds the order, and returns JSON data. The gateway receives the response and forwards it to the user.

This pattern mirrors familiar web development. Express.js, Spring Boot, and Flask all implement synchronous request handlers by default. HTTP and gRPC support this model natively. Most developers write their first web service using synchronous communication.

The Simplicity Advantage

Synchronous calls are straightforward to implement and reason about. Code reads sequentially. Fetch user data, then fetch their orders, then return both. The execution path matches the code structure.

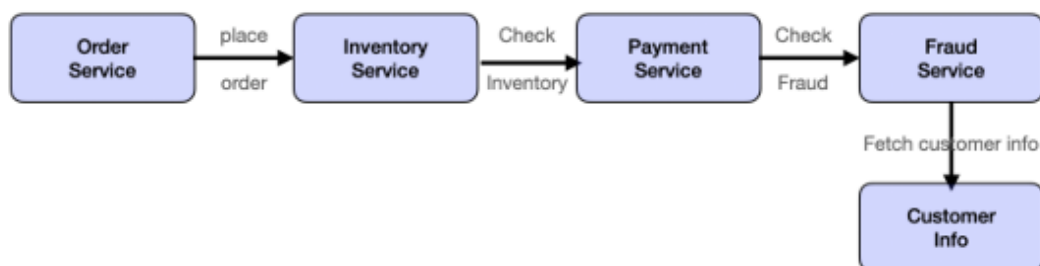
Debugging follows the same linear flow. Request logs show the entire chain. A failed request at timestamp 10:05:23 shows calls from the gateway to the order service to the database. Each step has clear input and output. Postman and curl let developers test endpoints directly.

Errors surface immediately. A 404 means the resource does not exist. A 500 indicates a server error. The client knows the operation failed and can retry or show an error message. No ambiguity exists about operation status.

The Coupling Problem

Synchronous calls create runtime dependencies. Service A cannot function if Service B is down. A user views their profile. The profile service calls the achievements service to fetch badges. If achievements is offline, the entire profile request fails. The user sees an error instead of their profile, even though their basic user data is available.

This coupling becomes worse with service chains. Service A calls B. B calls C. C calls D. If D responds in 200ms, C adds 50ms of processing, B adds 50ms, and A adds 50ms, total latency reaches 350ms. If D slows to 2 seconds, the entire chain waits 2 seconds plus overhead.

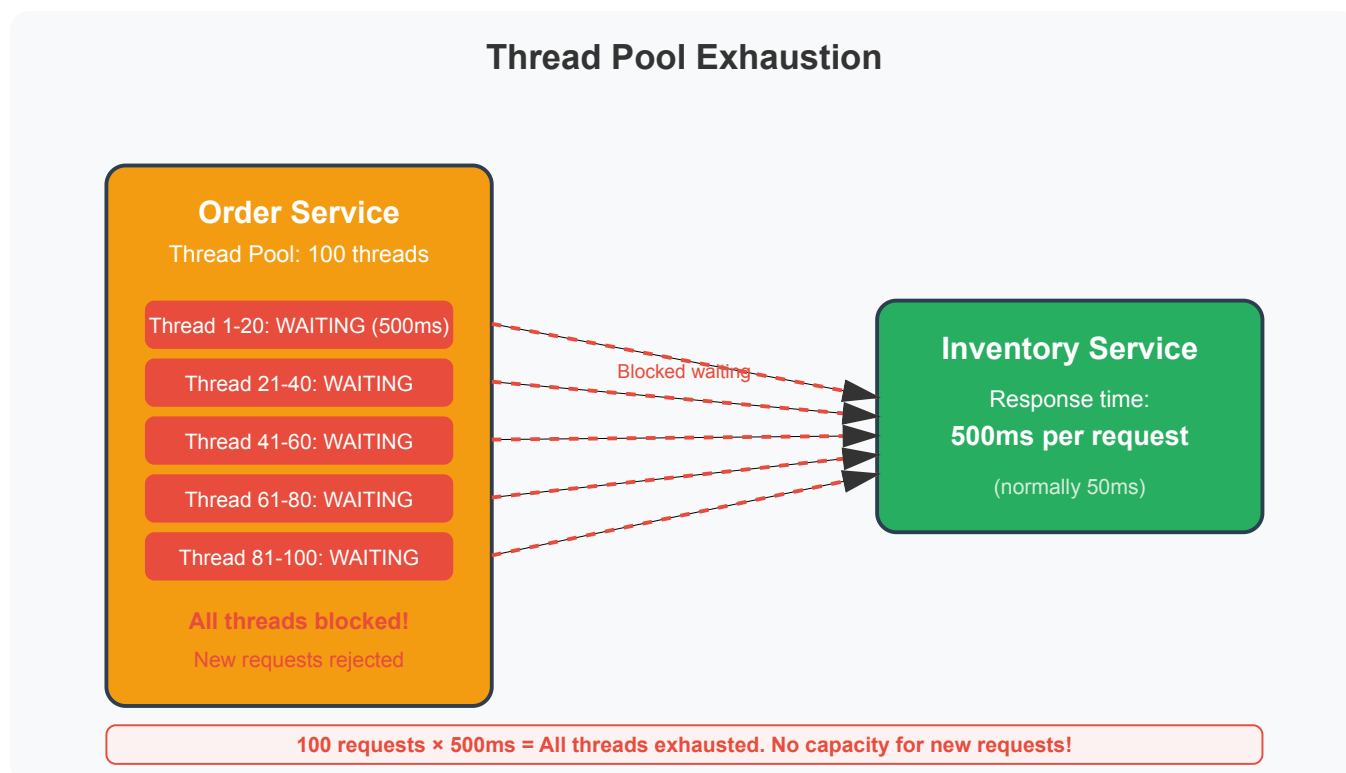


Failures cascade. If C crashes, B cannot get a response. B returns an error to A. A returns an error to the user. One service failure breaks three services. The blast radius expands with chain length.

Resource Contention

Each synchronous request ties up resources while waiting. A thread handles the request and blocks until the response arrives. Under high load, all available threads can be waiting for responses. New requests queue up or get rejected.

An order service handles 100 concurrent requests. Each request calls the inventory service and waits 500ms for a response. All 100 threads are blocked waiting. A new request arrives but no threads are available. The request fails or waits in a queue.



This resource exhaustion spreads upstream. If 10 gateway instances each have 100 threads, and all block waiting for the order service, the gateway cannot serve new requests. Clients see timeouts.

When Synchronous Communication Works

Use synchronous calls when you need an immediate response to proceed. User authentication must validate credentials before allowing access. Retrieving data for display requires the actual data. Payment processing needs confirmation before completing a purchase.

Keep service chains short. One or two hops are manageable. Three or more create latency and failure risks. If you find yourself chaining four services, consider whether all that data is truly needed immediately or if some can be fetched asynchronously.

Simple architectures with a few services handle synchronous calls well. A small application with three microservices can use REST APIs without complex failure handling. As the system grows, you need [timeouts](#), [retries](#), [circuit breakers](#), and [fallbacks](#) to handle the inevitable

failures.

The [implementation section](#) covers REST and gRPC in detail. The [failure handling section](#) explores patterns to make synchronous communication reliable at scale.